



**PONTIFÍCIA UNIVERSIDADE CATÓLICA DE CAMPINAS**

Engenharia de Computação — Projeto de Sistemas Digitais

## **Laboratório 11**

*Debouncer de Botão com Máquina de Estados Finita em VHDL*

*João Paulo Nunes Andrade - 25002703*

Campinas

Junho de 2025



# 1. Introdução

Botões mecânicos produzem ruído elétrico transitório ao serem pressionados ou soltos — um fenômeno chamado bouncing. O sinal resultante oscila rapidamente entre níveis lógicos por alguns milissegundos antes de se estabilizar, o que pode ser interpretado pelo circuito digital como múltiplos eventos de pressão. O objetivo deste laboratório é projetar um debouncer em VHDL usando uma Máquina de Estados Finita (FSM) de dois estados para filtrar esse ruído.

A abordagem escolhida é inteiramente temporal: o circuito só aceita a transição de estado quando o sinal do botão permanece estável por um número suficiente de ciclos de clock, determinado pelo contador interno.

## 2. Descrição do Problema

O sinal de entrada Botao é ativo em nível baixo: '0' indica botão pressionado, '1' indica solto. A saída Botao\_limpo deve refletir o estado real do botão apenas após a estabilização do sinal, eliminando pulsos espúrios causados pelo bouncing mecânico.

Os requisitos definidos pelo enunciado são:

- FSM com exatamente dois estados: SOLTO e PRESSIONADO
- Contador interno do tipo integer range 0 to 999999, zerado ao entrar em qualquer estado
- Reset assíncrono e processo sensível ao clock
- Limiar de 999999 ciclos para confirmar transição de estado

## 3. Análise e Derivação Lógica

### 3.1 Diagrama de Estados

A FSM possui dois estados estáveis. As transições dependem da combinação do sinal Botao com a condição contador  $\geq$  LIMAR. O contador só é zerado ao entrar num dos dois estados; enquanto o circuito permanece no mesmo estado aguardando estabilização, o contador continua incrementando a cada ciclo de clock.

#### **Resumo das transições:**

- SOLTO  $\rightarrow$  PRESSIONADO: Botao='0' AND contador  $\geq$  999999
- PRESSIONADO  $\rightarrow$  SOLTO: Botao='1' AND contador  $\geq$  999999



- Qualquer outra condição: permanece no estado atual, incrementa ou zera contador

### 3.2 Tabela de Transições de Estado

A tabela abaixo descreve o comportamento completo da FSM. 'X' indica condição irrelevante para a transição em questão.

| Estado atual | Botao      | Contador >= Limiar | Próximo estado | Saída (Botao_limpo) |
|--------------|------------|--------------------|----------------|---------------------|
| SOLTO        | 1 (solto)  | X                  | SOLTO          | 0                   |
| SOLTO        | 0 (press.) | 0                  | SOLTO          | 0                   |
| SOLTO        | 0 (press.) | 1                  | PRESSIONADO    | 0 → 1               |
| PRESSIONADO  | 0 (press.) | X                  | PRESSIONADO    | 1                   |
| PRESSIONADO  | 1 (solto)  | 0                  | PRESSIONADO    | 1                   |
| PRESSIONADO  | 1 (solto)  | 1                  | SOLTO          | 1 → 0               |

### 3.3 Lógica Booleana da Saída

A saída Botao\_limpo é função direta do estado atual — sem dependência das entradas. Trata-se de uma saída de Moore:

```
Botao_limpo = '1'   iff  estado_atual = PRESSIONADO
Botao_limpo = '0'   iff  estado_atual = SOLTO
```

Em VHDL, isso é implementado como atribuição condicional fora do processo (lógica combinacional pura), garantindo que a saída seja atualizada imediatamente ao mudar de estado, sem latência adicional de um ciclo de clock.

### 3.4 Cálculo do Limiar de Estabilização

Para uma placa com clock de 50 MHz, o tempo de estabilização correspondente a 999999 ciclos é:

$$T = 999999 / 50.000.000 \text{ Hz} \approx 20 \text{ ms}$$

20 ms é um valor padrão na literatura para debounce de botões mecânicos. Garante



que bounces típicos (que duram entre 1 ms e 10 ms) sejam completamente descartados.

## 4. Implementação VHDL

### 4.1 Arquitetura do Projeto

O projeto foi dividido em dois módulos distintos, seguindo separação de responsabilidades:

- debouncer\_fsm.vhd — núcleo da FSM: controla o estado, o contador e produz Botao\_limpo
- top\_level.vhd — instancia o debouncer\_fsm e conecta ao LED da placa (integração com o lab 10)

Um terceiro arquivo, tb\_debouncer\_fsm.vhd, fornece o testbench para verificação funcional por simulação.

### 4.2 Módulo debouncer\_fsm

O processo é sensível a Clock e Reset. O reset assíncrono é avaliado primeiro no if; apenas na ausência de reset o código avança para o bloco elsif rising\_edge(Clock). Isso garante que o reset aja independentemente do clock, conforme exigido.

Dentro do processo, a instrução case ramifica pelo estado atual. No estado SOLTO, quando Botao='0', verifica-se o limiar antes de transicionar; caso contrário, incrementa o contador. Se Botao='1' (solto), o contador é zerado — qualquer pulso de ruído no nível '1' reinicia a contagem, o que é o comportamento correto: só confirma transição quando '0' for estável o suficiente.

A lógica do estado PRESSIONADO é simétrica: aguarda Botao='1' estável para voltar ao SOLTO. A saída Botao\_limpo é atribuição concorrente fora do processo, caracterizando saída de Moore.

### 4.3 Módulo top\_level

O top\_level instancia debouncer\_fsm por nome (port map com named association), conectando os sinais da placa. A saída LED recebe diretamente botao\_debounced, mas esse ponto de conexão pode ser substituído pela lógica do lab 10 sem alterações no debouncer. A arquitetura foi declarada como structural, deixando claro que a entidade não contém lógica própria além da interconexão.



## 4.4 Testbench

O testbench `tb_debouncer_fsm` gera um clock de 20 ns (50 MHz) e aplica sequências de pulsos curtos intercalados (simulando bouncing) seguidos de um período longo estável. O limiar não foi reduzido no código sintetizável para não alterar o comportamento do hardware; a simulação representa o comportamento qualitativo da FSM nos primeiros ciclos, evidenciando que transições espúrias não atingem a saída `Botao_limpo`.

## 5. Estrutura de Entrega

Os três arquivos entregues são:

| Arquivo                           | Conteúdo                                                        |
|-----------------------------------|-----------------------------------------------------------------|
| <code>debouncer_fsm.vhd</code>    | FSM com dois estados, contador e saída <code>Botao_limpo</code> |
| <code>top_level.vhd</code>        | Integração estrutural do debouncer com a placa/LED              |
| <code>tb_debouncer_fsm.vhd</code> | Testbench para simulação funcional (não sintetizável)           |

## 6. Considerações Finais

A abordagem FSM com contador temporal é mais robusta do que soluções puramente combinacionais porque não depende de nenhum parâmetro analógico externo — o único parâmetro ajustável é o limiar, que se relaciona diretamente com o tempo de estabilização desejado. Trocar o clock da placa ou ajustar o limiar é suficiente para portar o debouncer para qualquer outra plataforma.

A separação entre `debouncer_fsm` e `top_level` mantém o módulo de debounce reutilizável: outros laboratórios que usem botões podem instanciar `debouncer_fsm` sem qualquer modificação, apenas substituindo a lógica de uso da saída `Botao_limpo` no `top_level`.